

Assignment 1: Introduction to ReactJS

Subject: ReactJS | Unit 1: Introduction to ReactJS

B.Sc. Computer Science

Q1. Simple Functional Component displaying "Hello React"

A Functional Component is the simplest type of React component. It is a plain JavaScript function that returns JSX. Below we create a component HelloReact and render it inside App.

Program / Code:

```
// App.js
import React from 'react';

// Functional Component
function HelloReact() {
  return (
    <div>
      <h1>Hello React</h1>
    </div>
  );
}

function App() {
  return (
    <div>
      <HelloReact />
    </div>
  );
}

export default App;
```

Expected Output:

Output:
Hello React

Q2. Class Component displaying Student Information

Class Components extend React.Component and must have a render() method that returns JSX. They are used when we need lifecycle methods or state (in older React code). Here we display student details.

Program / Code:

```
// App.js
import React, { Component } from 'react';

class StudentInfo extends Component {
  render() {
    return (
      <div>
        <h2>Student Information</h2>
        <p><b>Name    :</b> Rahul Patil</p>
        <p><b>Roll No :</b> 21</p>
        <p><b>Class   :</b> TY B.Sc. Computer Science</p>
        <p><b>College :</b> Solapur College</p>
      </div>
    );
  }
}

function App() {
  return <StudentInfo />;
}

export default App;
```

Expected Output:

Output:

```
Student Information
Name    : Rahul Patil
Roll No : 21
Class   : TY B.Sc. Computer Science
College : Solapur College
```

Q3. JSX Component with Heading and Paragraph

JSX (JavaScript XML) allows us to write HTML-like syntax inside JavaScript. React converts JSX to `React.createElement()` calls internally. Below component uses JSX to render a heading and paragraph.

Program / Code:

```
// App.js
import React from 'react';

function MyPage() {
  return (
    <div>
      <h1>Welcome to ReactJS</h1>
      <p>
        React is a JavaScript library for building user interfaces.
        It was developed by Facebook and is widely used for
        creating single-page applications (SPA).
      </p>
    </div>
  );
}

function App() {
  return <MyPage />;
}

export default App;
```

Expected Output:

```
Output:
Welcome to ReactJS
React is a JavaScript library for building user interfaces.
It was developed by Facebook and is widely used for
creating single-page applications (SPA).
```

Q4. JSX Expressions – Current Date and Time

In JSX, we can embed any valid JavaScript expression inside curly braces `{}`. Here we use `new Date().toString()` to display the current date and time dynamically when the component renders.

Program / Code:

```
// App.js
import React from 'react';

function DateTime() {
  const now = new Date();
  return (
    <div>
      <h2>Current Date and Time</h2>
      <p>Date : {now.toLocaleDateString()}</p>
      <p>Time : {now.toLocaleTimeString()}</p>
      <p>Full : {now.toString()}</p>
    </div>
  );
}

function App() {
  return <DateTime />;
}

export default App;
```

Expected Output:

Output:

Current Date and Time

Date : 08/03/2026

Time : 10:30:45 AM

Full : Sun Mar 08 2026 10:30:45 GMT+0530 (India Standard Time)

Q5. Parent and Child Components

In React, components can be nested inside other components. The outer component is called the Parent and the inner one is the Child. This is the foundation of React's component composition model.

Program / Code:

```
// App.js
import React from 'react';

// Child Component
function Child() {
  return (
    <div>
      <h3>I am the Child Component</h3>
      <p>This is rendered inside the Parent.</p>
    </div>
  );
}

// Parent Component
function Parent() {
  return (
    <div>
      <h2>I am the Parent Component</h2>
      <Child />
    </div>
  );
}

function App() {
  return <Parent />;
}

export default App;
```

Expected Output:

```
Output:
  I am the Parent Component
    I am the Child Component
      This is rendered inside the Parent.
```

Q6. Pass Data from Parent to Child using Props

Props (short for properties) allow data to flow from a parent component to a child component. Props are read-only – child components cannot modify the props they receive. We access props in functional components as a parameter.

Program / Code:

```

// App.js
import React from 'react';

// Child receives data via props
function Greeting(props) {
  return (
    <div>
      <h3>Hello, {props.name}!</h3>
      <p>You are {props.age} years old.</p>
      <p>City: {props.city}</p>
    </div>
  );
}

// Parent sends data as attributes
function App() {
  return (
    <div>
      <h2>Props Demo</h2>
      <Greeting name="Rahul" age={21} city="Solapur" />
      <Greeting name="Priya" age={20} city="Pune" />
    </div>
  );
}

export default App;

```

Expected Output:

Output:

Props Demo

Hello, Rahul! | You are 21 years old. | City: Solapur

Hello, Priya! | You are 20 years old. | City: Pune

Q7. Multiple Components Rendered in App Component

React applications are built by composing multiple components. The root App component can include Header, Body, Footer etc. This promotes reusability and separation of concerns.

Program / Code:

```
// App.js
import React from 'react';

function Header() {
  return <header><h1>My React Website</h1></header>;
}

function NavBar() {
  return <nav><p>Home | About | Contact</p></nav>;
}

function Content() {
  return (
    <main>
      <h2>Main Content Area</h2>
      <p>Welcome to my React application!</p>
    </main>
  );
}

function Footer() {
  return <footer><p>Copyright 2026 - ReactJS App</p></footer>;
}

function App() {
  return (
    <div>
      <Header />
      <NavBar />
      <Content />
      <Footer />
    </div>
  );
}

export default App;
```

Expected Output:

Output:
My React Website
Home | About | Contact
Main Content Area
Welcome to my React application!
Copyright 2026 - ReactJS App

Q8. State in a Class Component

State is a built-in object that stores dynamic data in a component. In class components, state is initialized in the constructor using `this.state = {}`. It is updated using `this.setState()`. When state changes, React automatically re-renders the component.

Program / Code:

```
// App.js
import React, { Component } from 'react';

class MyState extends Component {
  constructor(props) {
    super(props);
    // Initialize state
    this.state = {
      name: 'Rahul',
      score: 85,
      isPass: true
    };
  }

  render() {
    return (
      <div>
        <h2>State Demo</h2>
        <p>Name : {this.state.name}</p>
        <p>Score : {this.state.score}</p>
        <p>Result: {this.state.isPass ? 'Pass' : 'Fail'}</p>
      </div>
    );
  }
}

export default MyState;
```

Expected Output:

Output:

```
State Demo
Name : Rahul
Score : 85
Result: Pass
```


Q10. User Profile Component using Props

Props allow us to build reusable components. Here a UserProfile component accepts name, age, email, and role as props and renders a profile card. The same component can display different users just by passing different props.

Program / Code:

```
// App.js
import React from 'react';

function UserProfile(props) {
  return (
    <div style={{ border: '1px solid #ccc', padding: '10px',
      margin: '10px', borderRadius: '8px' }}>
      <h2>User Profile</h2>
      <p><b>Name :</b> {props.name}</p>
      <p><b>Age :</b> {props.age}</p>
      <p><b>Email :</b> {props.email}</p>
      <p><b>Role :</b> {props.role}</p>
    </div>
  );
}

function App() {
  return (
    <div>
      <UserProfile
        name="Rahul Patil"
        age={21}
        email="rahul@example.com"
        role="Student"
      />
      <UserProfile
        name="Prof. Sharma"
        age={38}
        email="sharma@college.edu"
        role="Teacher"
      />
    </div>
  );
}

export default App;
```

Expected Output:

```
Output:
[ User Profile Card ]
Name : Rahul Patil | Age: 21 | Email: rahul@example.com | Role: Student
[ User Profile Card ]
Name : Prof. Sharma | Age: 38 | Email: sharma@college.edu | Role: Teacher
```


Q11. Counter using State in a Class Component

State makes components interactive. Here we create a counter with Increment, Decrement and Reset buttons. Each button calls `setState()` to update the count, and React re-renders the UI automatically.

Program / Code:

```
// App.js
import React, { Component } from 'react';

class Counter extends Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 };
  }

  increment = () => {
    this.setState({ count: this.state.count + 1 });
  };

  decrement = () => {
    this.setState({ count: this.state.count - 1 });
  };

  reset = () => {
    this.setState({ count: 0 });
  };

  render() {
    return (
      <div>
        <h2>Counter App</h2>
        <h1>{this.state.count}</h1>
        <button onClick={this.increment}>Increment (+)</button>
        <button onClick={this.decrement}>Decrement (-)</button>
        <button onClick={this.reset}>Reset</button>
      </div>
    );
  }
}

export default Counter;
```

Expected Output:

Output:
Counter App
[0]
[Increment (+)] [Decrement (-)] [Reset]
(Clicking Increment shows 1, 2, 3...)

Q12. Convert Functional Component to Class Component

Any functional component can be converted to a class component by: 1) Extending `React.Component` 2) Wrapping JSX inside `render()` 3) Replacing function parameters with `this.props`. Below shows both versions side by side.

Program / Code:

```
// App.js
import React, { Component } from 'react';

// --- FUNCTIONAL COMPONENT (original) ---
// function Welcome(props) {
//   return <h2>Hello, {props.name}</h2>;
// }

// --- CLASS COMPONENT (converted) ---
class Welcome extends Component {
  render() {
    return (
      <div>
        <h2>Hello, {this.props.name}</h2>
        <p>Welcome to ReactJS Class Component.</p>
      </div>
    );
  }
}

function App() {
  return <Welcome name="Rahul" />;
}

export default App;
```

Expected Output:

```
Output:
Hello, Rahul!
Welcome to ReactJS Class Component.
```

Q13. Component Lifecycle – componentDidMount()

componentDidMount() is a lifecycle method that runs after the component is inserted into the DOM. It is the ideal place to fetch data from an API, set timers, or perform side effects. It runs only once after the first render.

Program / Code:

```
// App.js
import React, { Component } from 'react';

class LifeCycleDemo extends Component {
  constructor(props) {
    super(props);
    this.state = {
      message: 'Component is loading...',
      time: ''
    };
    console.log('1. Constructor called');
  }

  componentDidMount() {
    console.log('3. componentDidMount called');
    // Simulating data fetch after mount
    this.setState({
      message: 'Component Mounted Successfully!',
      time: new Date().toLocaleTimeString()
    });
  }

  render() {
    console.log('2. render() called');
    return (
      <div>
        <h2>Lifecycle Demo</h2>
        <p>{this.state.message}</p>
        <p>Mounted at: {this.state.time}</p>
      </div>
    );
  }
}

export default LifeCycleDemo;
```

Expected Output:

Console Output:

1. Constructor called
2. render() called
3. componentDidMount called
2. render() called (re-render after setState)

Browser Output:

Lifecycle Demo
Component Mounted Successfully!
Mounted at: 10:30:45 AM

Q14. Display a List of Subjects using JSX

In JSX, we can render lists using the JavaScript `.map()` method. Each item in the list must have a unique key prop so React can efficiently update the DOM when the list changes.

Program / Code:

```
// App.js
import React from 'react';

function SubjectList() {
  const subjects = [
    'ReactJS',
    'Data Structures',
    'Database Management',
    'Operating System',
    'Computer Networks',
    'Software Engineering'
  ];

  return (
    <div>
      <h2>My Subjects</h2>
      <ul>
        {subjects.map((subject, index) => (
          <li key={index}>{index + 1}. {subject}</li>
        ))}
      </ul>
    </div>
  );
}

function App() {
  return <SubjectList />;
}

export default App;
```

Expected Output:

Output:

My Subjects

1. ReactJS
2. Data Structures
3. Database Management
4. Operating System
5. Computer Networks
6. Software Engineering

Q15. JSX Expressions – Arithmetic Operations

JSX supports any JavaScript expression inside curly braces {}. This includes arithmetic calculations, string concatenation, ternary operators, and function calls. Below we demonstrate basic arithmetic operations rendered directly in JSX.

Program / Code:

```
// App.js
import React from 'react';

function ArithmeticOps() {
  const a = 25;
  const b = 10;

  return (
    <div>
      <h2>Arithmetic Operations in JSX</h2>
      <p>a = {a}, b = {b}</p>
      <table border="1" cellPadding="8">
        <tbody>
          <tr><td>Addition</td>      <td>{a} + {b} = {a + b}</td></tr>
          <tr><td>Subtraction</td>  <td>{a} - {b} = {a - b}</td></tr>
          <tr><td>Multiplication</td><td>{a} x {b} = {a * b}</td></tr>
          <tr><td>Division</td>      <td>{a} / {b} = {a / b}</td></tr>
          <tr><td>Modulus</td>       <td>{a} % {b} = {a % b}</td></tr>
          <tr><td>Power</td>         <td>{a} ^ {b} = {Math.pow(a, b)}</td></tr>
        </tbody>
      </table>
    </div>
  );
}

function App() {
  return <ArithmeticOps />;
}

export default App;
```

Expected Output:

```
Output: (a=25, b=10)
Addition      : 25 + 10 = 35
Subtraction   : 25 - 10 = 15
Multiplication : 25 x 10 = 250
Division      : 25 / 10 = 2.5
Modulus       : 25 % 10 = 5
Power         : 25 ^ 10 = 95367431640625
```

End of Assignment 1 — Introduction to ReactJS All programs are based on Unit 1 syllabus topics:
JSX, Components, State, Props, and Component Lifecycle.